

Reporte evaluación 1

Semestre: 4to.

Nombre: Yael Alexander Calderon Reyes.

Carrera: Ingeniería en Informática.

Numero de control: 242310752.

Nombre docente: Jesús Salas.

Fecha: 03 de junio del 2026.

INTRODUCCION

La administración de datos moderna se cimenta en la capacidad de transformar registros planos en activos de información con valor estratégico. El presente informe documenta el desarrollo de un ecosistema de software diseñado para la gestión de activos mediante archivos de organización secuencial.

A diferencia de los sistemas que dependen de motores de bases de datos preconfigurados, esta solución aborda la arquitectura desde su nivel más fundamental: la manipulación de flujos de texto (Streams) y la persistencia física en disco. A lo largo de este documento, se detallará cómo la integración de lenguajes como Python y PHP permite construir un pipeline de datos completo, que abarca desde la generación algorítmica de datasets masivos hasta la implementación de motores de búsqueda por filtrado secuencial.

SISTEMA DE GENERACION DE MASIVA Y FILTRADO SECUENCIAL

A. Descripción Detallada de la Solución

Este módulo constituye el núcleo de generación de datos del portafolio. Se desarrolló un motor de simulación en Python capaz de crear un Archivo Maestro de 500 registros con una estructura lógica definida. El sistema no solo genera datos aleatorios, sino que aplica reglas de negocio para asegurar que cada registro (representando modelos de vehículos a escala) posea atributos coherentes como ID, modelo, color, material, chasis y año.

La interfaz de usuario, desarrollada en una arquitectura web (HTML5/PHP), permite interactuar con este archivo de texto plano para realizar consultas dinámicas. El sistema es capaz de leer el archivo físico, identificar patrones de coincidencia y segmentar la información en un archivo de resultados temporal, cumpliendo con los estándares de procesamiento por lotes (Batch Processing).

B. Fundamentos de Ingeniería: Organización Secuencial con Delimitadores

Para este proyecto se seleccionó una organización de archivos basada en Delimitadores de Carácter (|), una técnica de ingeniería que optimiza la portabilidad:

Complejidad de Almacenamiento: El uso de delimitadores reduce el "overhead" de almacenamiento en comparación con formatos de ancho fijo, permitiendo que el archivo maestro.txt sea extremadamente ligero y fácil de transmitir a través de redes con ancho de banda limitado.

Algoritmo de Filtrado: El script procesar.php implementa un algoritmo de búsqueda lineal. Dado que los archivos planos carecen de un índice físico, el sistema optimiza la búsqueda utilizando la función stripos, la cual realiza una comparación binaria a nivel de stream, permitiendo una búsqueda flexible que ignora la distinción entre mayúsculas y minúsculas (Case Insensitive).

Persistencia Atómica: La generación de datos en Python utiliza buffers de escritura controlados para asegurar que, en caso de un fallo energético, el archivo no quede en un estado corrupto, garantizando que cada línea escrita sea un registro completo.

C. Análisis de la Arquitectura de Flujo

Capa de Datos: El archivo maestro.txt actúa como la única fuente de verdad (Single Source of Truth).

Capa de Procesamiento: PHP actúa como el motor de transformación, segmentando las cadenas de texto mediante la función `explode()`, la cual reconstruye la matriz de datos original en tiempo de ejecución.

Capa de Presentación: El uso de CSS profesional asegura que los datos, aunque provengan de un archivo de texto rudo, se presenten en una tabla jerárquica que facilita la auditoría de información.

MATRIZ DE EVALUACION

Atributo de Ingeniería	Implementación Técnica	Beneficio Operativo
Generación Algorítmica	Script Python con lógica de aleatorización controlada.	Capacidad de escalar de 100 a 1,000 registros sin modificar el código core.
Parsing de Datos	Función <code>explode</code> y manejo de punteros en PHP.	Transformación eficiente de texto plano a estructuras de memoria volátil.
Diseño de Interfaz	Frontend desacoplado con CSS lineal.	Garantiza una experiencia de usuario (UX) limpia y enfocada en la legibilidad de datos.
Persistencia Temporal	Generación de <code>filtrado.txt</code> por cada consulta.	Permite mantener un historial de la última búsqueda sin sobrecargar el archivo maestro.

Conclusión

El desarrollo de este proyecto permite concluir que el dominio de los archivos de organización secuencial es la competencia fundamental sobre la que se construye cualquier sistema de información robusto. A través de la implementación de un motor de generación en Python y una interfaz de procesamiento en PHP, se validó que es posible gestionar volúmenes de datos significativos (500 registros) manteniendo un control absoluto sobre el uso de recursos del sistema operativo.

La experiencia técnica obtenida demuestra que la eficiencia no reside únicamente en la potencia del lenguaje, sino en la correcta selección del formato de persistencia. El uso de archivos planos con delimitadores no solo garantiza la interoperabilidad entre diferentes plataformas, sino que obliga al ingeniero a comprender la mecánica subyacente de la lectura de flujos (Streams), los punteros de memoria y la lógica de filtrado algorítmico. Este

proyecto no solo cumple con los requisitos funcionales de captura y visualización, sino que establece un estándar de diseño modular que sirve como pilar para la transición hacia estructuras de datos más complejas y sistemas de bases de datos de nivel empresarial.